

科目：データベース演習

第2回：情報システムの構築とデータベース設計

講師：鈴木源吾

前回の復習

1. ガイダンス

- 進め方・評価
- 教科書・参考書について

2. PostgreSQLの動作確認

3. PostgreSQLに関する補足解説

4. pgAdminでメタデータを検索する

進め方のイメージ（第1回の補足）

今後の計画

- シラバス上は、概念設計→論理設計→物理設計という順だったが、まずはデータベースに慣れるために、概念設計を後に回す.
- データベースとして、PostgreSQLを使う予定だったが、より簡易で扱いやすいSQLiteもとりにあげていく
- Google Colabを使った作業が親しみやすいと思われるので、積極的に採用していく.

今回のトピック

1. 解説：システム開発とデータベース設計
 - システム開発の工程
 - 設計工程とデータベース
2. 例題：Google Colabを使ったデータベース作成・検索
3. 解説：データベースの設計
4. 演習：大学データベースの作成

topic 1

解説：システム開発とデータベース設計

システム開発の工程

ここでは、概要を説明する。詳細は「システム開発技術」で詳しく学ぶ
システム開発では、非常に大きなシステムを作ることが多い

→ 建物や橋などの建造物を作ることと少し似ている

- 無計画にすることはできない。とりあえず作る・・・ではうまくいかない
- 作る前に「設計」する必要がある
 - 作り始める前に、どうするかを決める
- 多くの人に関わるから、仕事を分類し、役割を決める必要がある

→ 開発をいくつかの工程（ステップ）に分解する必要がある

システム開発の工程

典型的な、システム開発の工程は以下である。基本的には、1.から4.の順で取り組む

- 工程の進め方は「システム開発技術」で詳しく学ぶ
- 注：用語はいろいろある。ここでは参考書の用語を用いた

1. 要件定義
2. 設計
3. 実装
4. テスト

システム開発の工程：要件定義

システムが満たすべき機能やサービスの水準である「要件」を，システム開発を依頼・発注している，利用者部門・顧客企業などと調整・合意して決める工程．アウトプットは要件定義書となる

例）オンラインゲームのユーザ管理システムを開発する．例えば，以下の要件を決める必要がある

- ユーザの数は最大何人を想定するか
- ユーザのどのような属性（プレイ回数，課金回数，等）を管理するか

システム開発の工程：設計

設計

定義された要件を満たすために必要なシステムを作るための、さまざまな設計（デザイン）を行う工程。アウトプットは設計書となる。

- 建築で言えば、実際に作る前に図面を引く作業
- システムモデリングで学んだUMLも用いられる
- アプリケーション設計，データ設計，ユーザーインターフェース設計，等

システム開発の工程：実装・テスト

実装

設計書にしたがってシステムを実際に作る工程

- 「実装する (implement)」という言い方をする
- プログラムを作成するコーディングをさすことが多いが、サーバーやネットワーク機器と言ったハードウェアの環境構築も含まれる

テスト

実装によって組み上がったシステムが、本当に実用に耐える品質であるかを試験（テスト）する工程

- 機能的な品質に対するテストと、性能や信頼性といった非機能品質のテストに分けられる

設計工程とデータベース

- 設計工程では、アプリケーション設計・データ設計・ユーザーインターフェース設計、などが含まれていた
- データ設計には、データベースに保持するデータの設計である「データベース設計」が含まれる
- データベースのデータは、様々なプログラムから共通に利用される、よって、データベースをしっかりと設計しておくことは非常に重要である

DOAとPOA

近年のソフトウェア開発では、**データ中心アプローチ**（DOA）という考え方が主流

- システムを作る際に、プログラムよりも前にデータの設計から始める方法論

その逆は、**プロセス中心アプローチ**（POA）と呼ばれる

- プロセス＝処理＝プログラムを先に設計する方法論

なぜDOAが主流なのか？

- プログラムの都合でデータを作ってしまうと、複数のプログラムで個別にデータを持って冗長になったり、矛盾が生じて、設計での誤りや手戻りが起きやすい

→ **システムを考えるときは、まずは、そのデータを整理してみよう**

topic 2

例題：Google Colabを使ったデータベース作成・検索

配布したノートブックで進める

topic 3

解説：データベースの設計

データベースの設計プロセス

データベースの基礎で、概念設計→論理設計→物理設計のステップで行うと学んだ

- 抽象→具体, 業務→処理, と段階的に行うことが有効
- 各設計段階でやることは, 人やプロジェクトによって変わることは多い

概念設計と正規化

【復習】 正規化：更新時異状が発生しないようなりレーションの分解

どこで正規化を行うか？（第1～3正規化）

→ 基本的には概念設計の段階で行われるべき

- エンティティが正規形でない場合は、正規化すること
- ただし、エンティティの抽出の段階で、繰り返しは別エンティティとしているため、第1正規形は実現されているはず
- さまざまな概念をエンティティとして分離して抽出していれば、自然と正規形に近い形にはなっていく

論理設計と物理設計

論理設計・物理設計でやるべきことの基本を演習し，データベースの作成までを行う

1. テーブル・カラムの決定
2. データ型の決定
3. 参照整合性の決定（主キー・外部キーの設定）
4. テーブルの作成
5. インデックスの作成
6. データの作成

今回は，1-4までを実施する

カラムのデータ型はなぜ必要か？

1. SQL文の中で関数を適切に利用するため

- SQLの機能である、文字列・数値・日付を処理する関数を利用できる。検索結果に対してプログラムで処理することもできるが、SQLの関数を利用する方が、簡潔に書けることが多い

2. SQL文のソート（order by句）で適切な結果を得るため

- 数値は数値の順番、文字は文字の順番で並べたい
 - 数値の昇順の例：1,20,110
 - 文字列の昇順の例：1,110,20

3. DBMSからプログラミングに値が渡されるときに、適切なデータ型で処理ができるようにするため

- ここはドライバの仕様にも依存する（最悪、全部文字列もありえる）

PostgreSQLの代表的なデータ型

PostgreSQLのデータ型の選択

- 連番型：データベースで独自に付与する識別番号（主キーとして使うIDなど）には、自動的に重複しない数値を割り当ててくれる、連番型のserialを使うのが便利（ただし、学籍番号のような既存の識別番号には使えない）
- 数字：整数値とする場合は通常はintegerでよいが、大きな値を使うときはbigintの利用の検討も必要。小数点を含む場合はdouble precisionでもよいが、計算における精度の正確性が必要なときはnumericを用いる必要がある
 - 固定長で計算しない数字（例：学籍番号・口座番号）は文字列の型でもよい
- 文字列：text一択でも問題ないと言われている
 - 分類や区分を表している場合は、単語をそれぞれ数値に対応させて、数値のデータ型を用いても良い。数値とすると単語の表記揺れを抑止できる
 - 性能やデータ格納などを配慮した詳細は、次ページ参考情報3.を参照せよ

PostgreSQLのデータ型の選択（続き）

- 日付/時刻：ログイン時刻などはtimestampを使う必要がある。日付まででよいのならdateを使う。アクセス間隔などではintervalを使う

PostgreSQLのデータ型に関する参考情報

1. PostgreSQL マニュアル 第8章 データ型

<https://www.postgresql.jp/document/16/html/datatype.html>

2. 【初心者向け】 PostgreSQLで考えるデータ型の種類とは？

<https://eng-entrance.com/postgres-sql-data-type>

3. Let's POSTGRES, 文字列型の使い分け

<https://lets.postgresql.jp/documents/technical/text-processing/1>

SQLiteの主なデータ型（分類と説明）

- **INTEGER**

整数型。主キーにすると自動採番も可能。

- **REAL**

浮動小数点型（8バイトの倍精度）。

- **TEXT**

文字列型。長さ制限なし、UTF-8/UTF-16で格納。

- **BLOB**

バイナリ型。画像やファイルをそのまま保存。

- **NUMERIC**

数値・日付・論理値など。実体はTEXTやREALになることも。

SQLiteの型の特徴（補足）

- 厳密な型チェックはせず、**型の傾向: affinity**で扱う。
- 型名が違ってても同じ分類になることがある
（例： `SMALLINT` , `BIGINT` → `INTEGER` ）
- 型名はあくまで目安で、柔軟に処理される。

SQLによるテーブル作成

CREATE TABLE文によりテーブルを作成できる。例えば、下図のテーブルを作成するSQL文をその下に記述した。テーブル・カラム・データ型を指定する

SQLによる主キー・外部キーの設定

- CREATE TABLEにおいて、主キーと外部キーを以下のように指定できる
 - 参照される側のテーブル（例：科目）は、参照する側のテーブル（例：実習課題）よりも先に作成されている必要があるので注意すること
- 外部キーを指定する効果：主キーにない値を外部キーに作成するとエラーとなる
 - 例）科目テーブルの科目IDに存在しない値を実習課題の科目IDに入れようとするとSQL実行時にエラーとなる
 - このような制約を外部キー制約、または、参照整合性制約と呼ぶ

テーブルの修正

既にテーブルを作った後で、仕様の変更・ミスの発見などの理由で、テーブルを修正したいときがある

- テーブル定義の変更：ALTER TABLE (DB基礎 第5回資料 p.34)
- テーブルを削除して再度作成：DROP TABLE (DB基礎 第5回資料 p.33) + (再度) CREATE TABLE

という、2つの方法があるが、今回の演習のような初期構築の場合は、テーブル作成用のSQL文の修正も結局必要になるので、後者の削除して作り直しの方が（手数は多いものの）やりやすいだろう

注意：REFERENCES句を使用して、外部キー制約を設定する場合、参照されるテーブルを削除しようとするエラーとなる。参照するテーブルから順に削除すること

topic 3

演習：大学データベース作成

大学データベースについて

- 本学は、ホームページで教員情報を公開している
 - <https://kaishi-pu.ac.jp/department/teacher-all/>
- 現在は、データベースを使用せず、HTMLなどの表示情報に直接、教員の氏名などの情報が書き込まれている
- 今後の更新の容易性や、検索の柔軟性などのために、データベースを導入し、そのデータから表示情報を生成するようにしたい

どんなデータベースを作成したらよい？

- どんなテーブルが必要か？
- どんなカラムが必要か？

演習：大学データベースの作成

- 別途配布するテーブル情報を参照し，配布したColabノートブックを用いて，2つのテーブルを作成し，データ（行）を挿入せよ
- SQLiteのメモリデータベースで作成すること
- 自分の学籍番号の下一桁で決まる学部についてデータを作成すること
 - 学籍番号の下一桁が0, 3, 6, 9：事業創造学部（教員19人）
 - 学籍番号の下一桁が1, 4, 7：情報学部（教員15人）
 - 学籍番号の下一桁が2, 5, 8：アニメ・マンガ学部（教員18人，教育助手除く）
- そのノートブックをリンクで提出すること